

Rapport de stage: Imagerie

Olivier

2025

Table des matières

1	Détection des contours	2
1.1	Définitions et terminologie	3
1.2	Mesure de l'efficacité d'une méthode	5
1.3	Convolution : définition	6
1.4	Filtres	7
1.5	Ordre 1	9
1.5.1	Approximations des dérivées partielles	9
1.5.2	Mise en pratique	11
1.5.3	Problème sensibilité au bruit	14
1.6	Ordre 2	17
1.6.1	Dérivées secondes selon les gradients	17
1.6.2	Non annulation des déterminants des Hessiennes	20
1.6.3	Valeurs propres de la Hessienne	23
1.6.4	Ecart entre déterminant et trace de la Hessienne	23
1.6.5	Changement de signe du Laplacien	23
1.7	Ordre 3	24
1.8	D'autres filtres	25
1.8.1	Filtres sans nom 1	26
1.8.2	Filtre de Sobel	28
1.8.3	Filtre de Prewitt	28
1.9	temp	30
1.10	Transformée de Radon	31
1.11	Interpolation	32
1.12	Lignes de niveau et de courant	33
1.13	Transformée de Fourier	34
1.13.1	Continue 1D	34
1.13.2	Discret 1D	34
1.13.3	Continue 2D	35
1.13.4	Discret 2D	35
1.14	Retour sur la convolution	38
1.15	Convolution non linéaire	39
1.16	Transformée d'ondelettes	40
1.16.1	Transformée d'ondelettes continue	40
1.16.2	Transformée d'ondelettes discrètes	40
1.17	A s'intéresser	41

Chapitre 1

Détection des contours

GitHub : <https://github.com/Liuuuv/DetectionContour>

1.1 Définitions et terminologie

Soit $f : \mathbb{R}^2 \rightarrow [0, 1]$ représentant une image **analogique**, i.e. f est continue.

En principe f est à valeurs dans $[0, 255]^3$ mais on peut se ramener à valeurs dans $[0, 1]$ en prenant la moyenne des composantes normalisées. Cela revient à considérer l'image en nuances de gris.

Dans la suite de ce chapitre, on suppose, sauf mention contraire, que f est à valeurs dans $[0, 1]$.

On suppose que f est nulle en dehors d'un rectangle $\mathcal{R} \subseteq \mathbb{R}^2$ de taille $(m, n) \in \mathbb{N}^2 \setminus \{(0, 0)\}$

Soit $M, N \in \mathbb{N}$. On échantillonne $\mathcal{R}$ par une famille $((x_k, y_l))_{(k,l) \in \llbracket 0, M-1 \rrbracket \times \llbracket 0, N-1 \rrbracket} \in (\mathbb{R} \times \mathbb{R})^{MN}$. On suppose sans perdre de généralité (quitte à dilater $\mathcal{R}$) :

$$\begin{cases} \forall k \in \llbracket 0, M-2 \rrbracket & |x_{k+1} - x_k| = 1 \\ \forall l \in \llbracket 0, N-2 \rrbracket & |y_{l+1} - y_l| = 1 \end{cases}$$

On peut alors voir une image comme $u = (u_{ij})_{(i,j)} \in \mathbb{R}^{\mathbb{Z} \times \mathbb{Z}}$ où :

$$\forall (i, j) \in \llbracket 1, n \rrbracket^2 \quad u_{ij} = f(x_i, y_j)$$

et les u_{ij} en dehors du carré C sont choisis nuls.

On dit qu'on a **numérisé** l'image analogique f et on a alors obtenu une image dite **numérique**.

On aurait aussi pu poser $u_{ij} = f(x_{i \bmod n}, y_{j \bmod n})$. Ou dupliquer les pixels des bords, ou étendre par symétrie.



FIGURE 1.1 – Image originale, taille : (573, 640)



FIGURE 1.2 – Image (sous-)échantillonnée, taille : (144, 160), 1 pixel sur 4 pris (noté $1/4$)



FIGURE 1.3 – Image échantillonnée (1/2)
en nuances de gris

1.2 Mesure de l'efficacité d'une méthode

L'objectif est de présenter différentes méthodes de détection de contours et de les comparer.

On dit qu'on a les contours d'une image si l'on a une image binaire (pixels noirs ou blancs).

Nous allons nous baser sur différents facteurs :

- "Résistance" au bruit : en pratique, les images auront du bruit.
- Localisation des contours : on cherche des contours les plus proches des vrais contours.
- Détection : on veut détecter tous les contours recherchés.
- Fausse détection : on ne veut pas que de faux contours soient détectés.

On se munit d'images de "test", ce sont des images où les contours sont connus. Elles permettent une référence. On note cette image `image_ref`.

On voit les images des matrices sur $\mathbb{R}$

[photo image de test]

On définit le **coût** d'une méthode comme suit :

On pose `image_diff` := contours – image_ref

Et on pose :

$$[\text{image_diff}^+]_{ij} = \begin{cases} [\text{image_diff}]_{ij} & \text{si } [\text{image_diff}]_{ij} > 0 \\ 0 & \text{sinon} \end{cases}$$

et

$$[\text{image_diff}^-]_{ij} = \begin{cases} -[\text{image_diff}]_{ij} & \text{si } [\text{image_diff}]_{ij} < 0 \\ 0 & \text{sinon} \end{cases}$$

Puis le coût de la méthode :

$$\text{coût} = w_0 \mathcal{S}(\text{image_diff}^+) + w_1 \mathcal{S}(\text{image_diff}^-)$$

où $\mathcal{S}$ renvoie la somme de tous les coefficients d'une matrice donnée.

1.3 Convolution : définition

Soient $u, h \in \mathbb{R}^{\mathbb{Z} \times \mathbb{Z}}$. On définit pour $(i, j) \in \mathbb{Z} \times \mathbb{Z}$:

$$[u * h]_{ij} = \sum_{k=-\infty}^{\infty} \sum_{l=-\infty}^{\infty} u_{i-k, j-l} h_{kl}$$

Souvent, u sera vu comme l'image et h ce qu'on appelle un filtre (voir section sur les filtres), de support souvent fini et très petit (en cardinal).

Il nous sera utile d'écrire la convolution comme suit :

$$[u * h]_{ij} = \sum_{(k,l) \in \text{Supp } h} u_{i-k, j-l} h_{kl}$$

où $\text{Supp } h := \{(k, l) \in \mathbb{Z} \times \mathbb{Z} \mid h_{kl} \neq 0\}$

On peut vérifier que $*$ est bilinéaire, associative et commutative, de neutre :

$$\delta_{0,0} := \left(\begin{cases} 1 & \text{si } i = j = 0 \\ 0 & \text{sinon} \end{cases} \right)_{(i,j) \in \mathbb{Z} \times \mathbb{Z}}$$

De manière analogue, on définit pour $(i, j) \in \mathbb{Z} \times \mathbb{Z}$:

$$\delta_{i,j} := \left(\begin{cases} 1 & \text{si } i = k \text{ et } j = l \\ 0 & \text{sinon} \end{cases} \right)_{(i,j) \in \mathbb{Z} \times \mathbb{Z}}$$

On notera les filtres par une matrice avec un coefficient encadré qui représentera la position $(0,0)$. On omettra d'écrire des coefficients nuls.

Exemple de filtre :

$$\begin{pmatrix} 1 & \boxed{-1} \end{pmatrix} = \delta_{-1,0} - \delta_{0,0}$$

Donc on a :

$$\forall u \in \mathbb{R}^{\mathbb{Z} \times \mathbb{Z}} \quad u * \begin{pmatrix} \boxed{1} \end{pmatrix} = u$$

Remarque 1.3.1. On observe que le filtre utilisé dans la convolution et le filtre vu en tant qu'humain sont symétrisés centralement d'un angle de π .

Remarque 1.3.2. Lorsqu'on parlera d'image normalisée, on sous-entendra pour la norme infinie.

1.4 Filtres

Définition 1.4.1. Un **filtre** est une application $\mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{R}$ à support fini. c'est la donnée d'une matrice (représentant le support) et d'un coefficient encadré pour symboliser le coefficient d'indice $(0,0)$, appelé le centre.

Définition 1.4.2 (Noyau). Soit h un filtre. On appelle **noyau** de h la plus petite matrice représentant le support de h

Exemple 1.4.1. Le filtre $\delta_{00} + 2\delta_{01}$ est de noyau $\begin{pmatrix} \boxed{1} & 2 \end{pmatrix}$ mais peut aussi être représenté par la matrice

$$\begin{pmatrix} 0 & 0 & 0 \\ 0 & \boxed{1} & 2 \\ 0 & 0 & 0 \end{pmatrix}$$

Remarque 1.4.1 (Encadrement omis). S'il n'y a pas d'ambiguïté, on omettra parfois d'encadrer le centre du filtre.

Remarque 1.4.2 (Filtre-Noyau). On identifiera parfois les filtres avec leur noyau.

Définition 1.4.3 (Dimension d'un filtre). On dit qu'un filtre est à **1 dimension** si son noyau est une matrice ligne ou une matrice colonne. Si cela n'est pas le cas, on dira que le filtre est à **2 dimension**.

Définition 1.4.4 (Filtre séparable). Soit h un filtre. On dit que h est **séparable** s'il est convolé de deux filtres à 1 dimension, i.e. s'il existe $h_1, h_2 : \mathbb{Z} \rightarrow \mathbb{R}$ tels que $h = h_1 * h_2$

Propriété 1.4.1 (Support des filtres séparables).

Soit h un filtre séparable. On prend h_1, h_2 deux filtres à 1 dimension vérifiant $h = h_1 * h_2$. Alors :

$$\# \text{Supp } h = \# \text{Supp } h_1 \times \# \text{Supp } h_2$$

Démonstration. Par récurrence sur la taille d'un des deux supports (l'autre fixé) □

Remarque 1.4.3 (Utilité des filtres séparables). Soit h un filtre et u une image numérique de taille $(M, N) \in \mathbb{Z}^2$. On suppose h séparable et on prend alors h_1, h_2 deux filtres à 1 dimension vérifiant $h = h_1 * h_2$. On rappelle que pour $(i, j) \in \llbracket 0, M-1 \rrbracket \times \llbracket 0, N-1 \rrbracket$, on a :

$$[u * h]_{ij} = \sum_{(k,l) \in \text{Supp } h} u_{i-k, j-l} h_{kl}$$

On observe que pour calculer $u * h$ directement, il faut effectuer $MN \# \text{Supp } h$ opérations.

Or pour calculer $u * h_1$, il faut effectuer $MN \# \text{Supp } h_1$ opérations.

De même, il faut effectuer $MN \# \text{Supp } h_2$ opération pour calculer $u * h_2$.

Donc calculer $u * h$ en utilisant h_1 et h_2 nécessite $MN(\# \text{Supp } h_1 + \# \text{Supp } h_2)$.

Donc il est (largement) préférable d'un point de vue temps de calcul d'utiliser des filtres séparables plutôt que des filtres non séparables.

Soient $h_1 = \begin{pmatrix} k_{-n} & \dots & \boxed{k_0} & \dots & k_n \end{pmatrix}$ et $h_2 = \begin{pmatrix} l_{-m} \\ \vdots \\ \boxed{l_0} \\ \vdots \\ l_m \end{pmatrix}$ deux filtres à 1 dimension. On observe :

$$h_1 * h_2 = \begin{pmatrix} l_{-m}k_{-n} & l_{-m}k_{-(n-1)} & \dots & l_{-m}k_n \\ l_{-(m-1)}k_{-n} & l_{-(m-1)}k_{-(n-1)} & \dots & l_{-(m-1)}k_n \\ \vdots & \vdots & \ddots & \vdots \\ l_mk_{-n} & \dots & \dots & l_mk_n \end{pmatrix}$$

et donc :

$$h_1 \star h_2 = \begin{pmatrix} l_{-m} \\ \vdots \\ l_m \end{pmatrix} \times (k_{-n} \quad \dots \quad k_n)$$

On déduit la propriété suivante :

Propriété 1.4.2. Soient $m, n \in \mathbb{N}$ Soit $h \in \mathcal{M}_{mn}(\mathbb{R})$ un filtre. Alors :

$$h \text{ est séparable} \iff \exists \begin{cases} U \in \mathcal{M}_{m1}(\mathbb{R}) \\ V \in \mathcal{M}_{n1}(\mathbb{R}) \end{cases} \quad h = UV^\top$$

Corollaire 1.4.1. Soient $m, n \in \mathbb{N}$ Soit $h \in \mathcal{M}_{mn}(\mathbb{R})$ un filtre. Alors :

$$h \text{ est séparable} \iff h \text{ est de rang 1}$$

Démonstration. ($\Leftarrow$) On écrit $h = (C_1 \quad \lambda_2 C_1 \quad \dots \quad \lambda_n C_1)$ et donc :

$$h = C_1 (1 \quad \lambda_2 \quad \dots \quad \lambda_n)$$

qui est bien le produit d'une matrice colonne avec une matrice ligne. Comme voulu. $\square$

Propriété 1.4.3. La matrice d'un filtre est tridiagonale de Toeplitz (périodique) par blocs (blocs de la taille de l'image)

Suite de filtres ?

approcher filtre par filtre séparables ? -> convoluer des filtres ? pdv matriciel !

les seuls filtres (continues) séparables isotropes sont les gaussiennes

exp est une somme de polynomes, on peut chopper une gaussienne comme ça ?

<https://naghemanth.medium.com/applications-of-linear-algebra-in-image-filters-part-ii-separable-filters-f4635ea9f94f>

convolution comme matrice, svd

1.5 Ordre 1

Dans une image, un contour est intuitivement une forte variation de la luminosité. Ainsi, on va essayer de les détecter en regardant les extremas des dérivés.

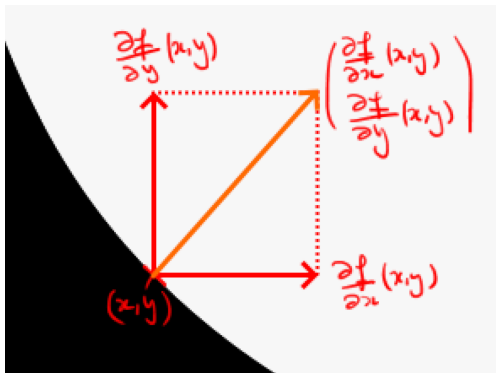


FIGURE 1.4 – Schéma du gradient de f en $(x, y) \in \mathbb{R}^2$

1.5.1 Approximations des dérivées partielles

Hypothèses : $\text{Im } f$ est un plan autour de chaque point d'échantillonnage au sens suivant :
Pour tout $(k, l) \in \llbracket 2, n-1 \rrbracket^2$, il existe $(\alpha, \beta) \in \mathbb{R}$ vérifiant

$$\forall x \in B_{\|\cdot\|_1}(x_k, y_l) \quad f(x) = f(x_k, y_l) + \alpha(x - x_k) + \beta(y - y_l)$$

Alors on peut écrire pour tout $(k, l) \in \llbracket 2, n-1 \rrbracket^2$:

$$\frac{\partial f}{\partial x}(x_k, y_l) = f(x_{k+1}, y_l) - f(x_k, y_l)$$

et

$$\frac{\partial f}{\partial y}(x_k, y_l) = f(x_k, y_{l+1}) - f(x_k, y_l)$$

(Différence finie décentrée à droite, notée DFD. On note de même DFG pour la gauche)

et donc on peut obtenir l'expression des dérivées partielles de f en les (x_k, y_l) (les notations sont abusives) :

$$\frac{\partial f}{\partial x}(x_k, y_l) = \left[u * \begin{pmatrix} 1 & \boxed{-1} \end{pmatrix} \right]_{k,l}$$

et

$$\frac{\partial f}{\partial y}(x_k, y_l) = \left[u * \begin{pmatrix} 1 \\ \boxed{-1} \end{pmatrix} \right]_{k,l}$$

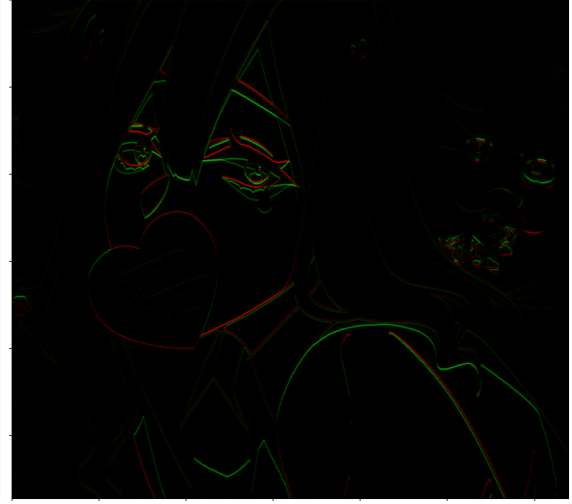
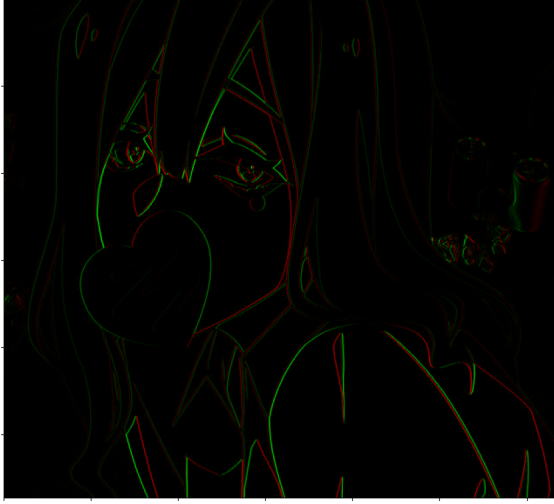


FIGURE 1.5 – Image normalisée de $\frac{\partial f}{\partial x}$ avec $\begin{pmatrix} 1 & -1 \end{pmatrix}$ FIGURE 1.6 – Image normalisée de $\frac{\partial f}{\partial y}$ avec $\begin{pmatrix} 1 \\ -1 \end{pmatrix}$

FIGURE 1.7 – En vert les dérivées positives, en rouge les négatives

De la même manière, on peut calculer quelques dérivées partielles secondes (en supposant que $\frac{\partial f}{\partial x}$ et $\frac{\partial f}{\partial y}$ soient des plans autour des points d'échantillonnage) :

$$\frac{\partial^2 f}{\partial x^2}(x_k, y_l) = \frac{\partial f}{\partial x}(x_k + 1, y_l) - \frac{\partial f}{\partial x}(x_k, y_l) \quad (\text{DFD})$$

$$= \left[u * \begin{pmatrix} 1 & -1 & 0 \end{pmatrix} \right]_{k,l} - \left[u * \begin{pmatrix} 0 & 1 & -1 \end{pmatrix} \right]_{k,l} \quad (\text{DFD})$$

$$\frac{\partial^2 f}{\partial x^2}(x_k, y_l) = \left[u * \begin{pmatrix} 1 & -2 & 1 \end{pmatrix} \right]_{k,l}$$

et de même (DFD, DFD) :

$$\frac{\partial^2 f}{\partial y^2}(x_k, y_l) = \left[u * \begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix} \right]_{k,l}$$

Mais on peut aussi considérer (on préférera cette version car applicable plus partout) :

$$\frac{\partial^2 f}{\partial x^2}(x_k, y_l) = \frac{\partial f}{\partial x}(x_k + 1, y_l) - \frac{\partial f}{\partial x}(x_k, y_l) \quad (\text{DFD})$$

$$= \left[u * \begin{pmatrix} 1 & -1 \end{pmatrix} \right]_{k,l} - \left[u * \begin{pmatrix} 1 & -1 \end{pmatrix} \right]_{k,l} \quad (\text{DFG})$$

$$\frac{\partial^2 f}{\partial x^2}(x_k, y_l) = \left[u * \begin{pmatrix} 1 & -2 & 1 \end{pmatrix} \right]_{k,l}$$

et de la même manière (DFD, DFG) :

$$\frac{\partial^2 f}{\partial y^2}(x_k, y_l) = \left[u * \begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix} \right]_{k,l}$$

On observe que peu importe les choix faits de décentrer à droite ou à gauche, on obtient les mêmes filtres à translation près. On préfère utiliser les filtres où leur centre est le plus "au centre" du support.

Le même principe nous donne un moyen de calculer $\frac{\partial^2 f}{\partial x \partial y}(x_k, y_l)$.
Voici un tableau de quelques filtres relatés aux dérivés :

$\frac{\partial f}{\partial x}$	$\begin{pmatrix} 1 & \boxed{-1} \end{pmatrix}$ (DFD)
$\frac{\partial f}{\partial y}$	$\begin{pmatrix} 1 \\ \boxed{-1} \end{pmatrix}$ (DFD)
$\frac{\partial^2 f}{\partial x \partial y}$	$\begin{pmatrix} 1 & -1 \\ -1 & \boxed{1} \end{pmatrix} = \begin{pmatrix} 1 & \boxed{-1} \end{pmatrix} * \begin{pmatrix} 1 \\ \boxed{-1} \end{pmatrix}$
$\frac{\partial^2 f}{\partial x^2}$	$\begin{pmatrix} 1 & \boxed{-2} & 1 \end{pmatrix} = \begin{pmatrix} 1 & \boxed{-1} \end{pmatrix} * \begin{pmatrix} \boxed{1} & -1 \end{pmatrix}$
$\frac{\partial^2 f}{\partial y^2}$	$\begin{pmatrix} 1 \\ \boxed{-2} \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ \boxed{-1} \end{pmatrix} * \begin{pmatrix} \boxed{1} \\ -1 \end{pmatrix}$
$\frac{\partial^3 f}{\partial x^2 \partial y}$	$\begin{pmatrix} 1 & -2 & 1 \\ -1 & \boxed{2} & -1 \end{pmatrix} = \begin{pmatrix} 1 \\ \boxed{-1} \end{pmatrix} * \begin{pmatrix} 1 & \boxed{-2} & 1 \end{pmatrix}$
$\frac{\partial^3 f}{\partial y^2 \partial x}$	$\begin{pmatrix} 1 & -1 \\ -2 & \boxed{2} \\ 1 & -1 \end{pmatrix} = \begin{pmatrix} 1 & \boxed{-1} \end{pmatrix} * \begin{pmatrix} 1 \\ \boxed{-2} \\ 1 \end{pmatrix}$

1.5.2 Mise en pratique

Intuitivement, un contour est un extremum local du gradient. Pour les trouver, on peut chercher les maximums de :

$$\begin{cases} \mathbb{R}^2 \rightarrow \mathbb{R} \\ (x, y) \mapsto \partial_{g_{x,y}} f(x, y) \end{cases}$$

Pour $(x, y) \in \mathbb{R}^2$, la dérivée de f selon $g_{x,y} := \frac{\nabla f(x, y)}{\|\nabla f(x, y)\|_2}$. L'expression de cette dérivée est :

$$\partial_{g_{x,y}} f(x, y) = \|\nabla f(x, y)\|_2$$

Donc le but est de maximiser en norme 2 le gradient.

Grâce au produit de convolution de filtres bien choisis, on a accès aux dérivés partielles de f sur l'échantillonnage. On peut donc calculer la norme 2 du gradient :

$$\|\nabla f\|_2 = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$



FIGURE 1.8 – Norme 2 du gradient (image normalisée). Avec $\begin{pmatrix} 1 & \boxed{-1} \end{pmatrix}$ et $\begin{pmatrix} 1 \\ \boxed{-1} \end{pmatrix}$

On peut aussi considérer la norme 1 du gradient :

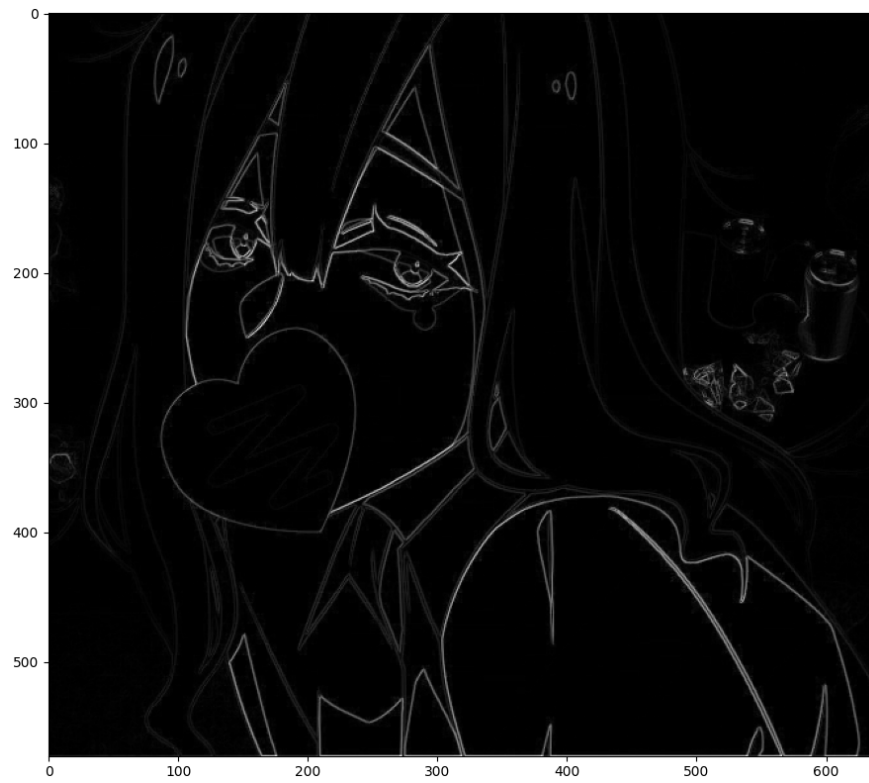


FIGURE 1.9 – Norme 1 du gradient. Avec $\begin{pmatrix} 1 & -1 \end{pmatrix}$ et $\begin{pmatrix} 1 \\ -1 \end{pmatrix}$

Maintenant, on peut fixer un seuil et affirmer qu'on est sur un contour si la valeur du pixel dépasse le seuil. Sinon, non.



FIGURE 1.10 – Seuil : 0.03 FIGURE 1.11 – Seuil : 0.065 FIGURE 1.12 – Seuil : 0.13

On peut aussi colorer les pixels en fonction des angles des gradients :

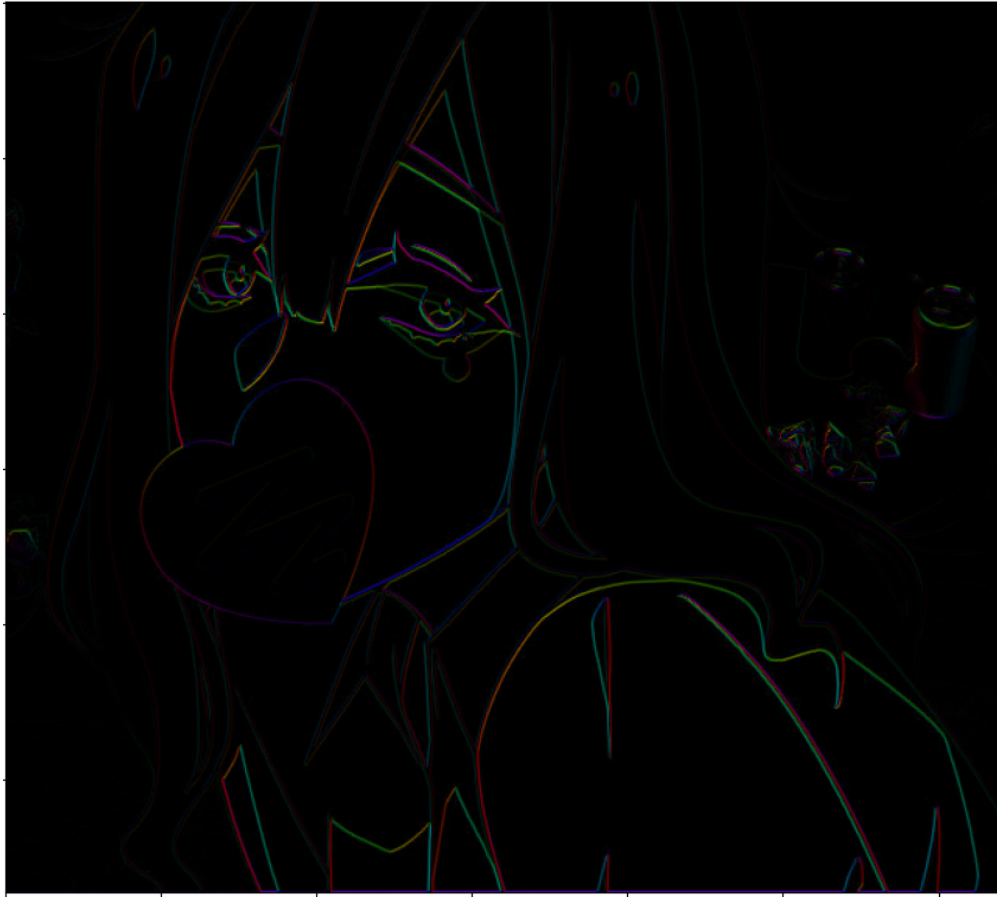


FIGURE 1.13 – Angles des gradients (filtre DFD)

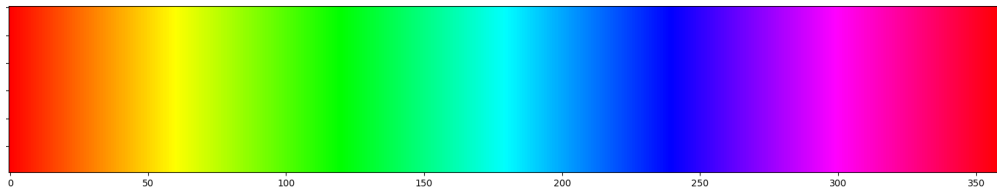


FIGURE 1.14 – Les degrés en abscisses

1.5.3 Problème sensibilité au bruit

Jusqu'ici, on a travaillé avec un dessin. Il n'y avait aucun bruit et les contours étaient clairs (marqués en noir). Voyons ce qu'il se passe si nous rajoutons du bruit à l'image :



FIGURE 1.15 – Sans bruit



FIGURE 1.16 – Avec bruit

FIGURE 1.17 – Bruit gaussien d'écart-type 0.1

Le bruit est accentué et les contours sont plus difficiles à percevoir. En appliquant des seuils on obtient :



FIGURE 1.18 – Sans bruit, seuil de 0.04 FIGURE 1.19 – Avec bruit, seuil de 0.35

FIGURE 1.20 – Bruit gaussien d'écart-type 0.1

1.6 Ordre 2

On suppose $f \in \mathcal{C}^2$

Étudier les images au second ordre permet une meilleure précision lors de la détection des contours ("courbes" plus fines et plus centrées sur le milieu des côtés). Mais les filtres utilisés sont très sensibles au bruit, ce qui ne les rend pas très utilisables.

1.6.1 Dérivées secondes selon les gradients

On sait qu'un contour se produit aux extremums des gradients, i.e. aux maximums des dérivées dans les sens des gradients. Mais alors cela revient à étudier les zéros des dérivées secondes dans les sens des gradients. Soit $(x, y) \in \mathbb{R}^2$. Calculons :

$$\underbrace{\partial_{g_{x,y}} \partial_{g_{x,y}} f(x, y)}_{:= \partial_{g_{x,y}}^2 f(x, y)} = \frac{\nabla f(x, y)^\top \text{Hess}_f(x, y) \nabla f(x, y)}{\|\nabla f(x, y)\|_2^3}$$

Que l'on peut réécrire :

$$\partial_{g_{x,y}}^2 f(x, y) = \left(\frac{\frac{\partial^2 f}{\partial x^2} \left(\frac{\partial f}{\partial x} \right)^2 + 2 \frac{\partial^2 f}{\partial x \partial y} \frac{\partial f}{\partial x} \frac{\partial f}{\partial y} + \frac{\partial^2 f}{\partial y^2} \left(\frac{\partial f}{\partial y} \right)^2}{\|\nabla f\|_2^3} \right) (x, y)$$

Puis regardons ses changements de signe.



FIGURE 1.21 – Dérivées secondes dans les sens des gradients, en vert lorsque c'est positif, en rouge sinon. seuil : 0.01

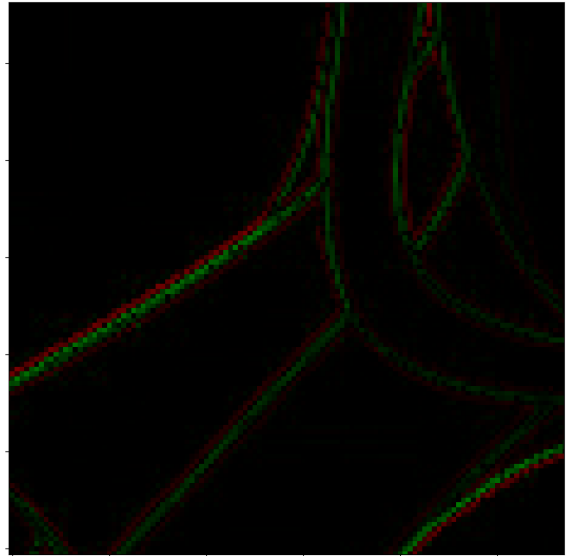


FIGURE 1.22 – Zoom

On observe que c'est positif et négatif le long des contours, ce qui est attendu au regard de la définition de $\partial_{g_{x,y}}^2 f(x, y)$ pour $(x, y) \in C$.

On peut donc appliquer un seuil au sens suivant :

$$\text{valeur du nouveau pixel} = \begin{cases} 1 & \text{si la valeur du pixel est au dessus du seuil} \\ -1 & \text{si la valeur du pixel est en dessous de l'opposé du seuil} \\ 0 & \text{sinon} \end{cases}$$

On obtient donc :



FIGURE 1.23 – Dérivées secondes dans les sens des gradients, en vert : 1, en rouge : -1 , seuil : 0.02

Puis en mettant un pixel blanc à chaque changement de signe (plus précisément, en regardant les 3 pixels en bas à gauche et en les mettant blancs si changement de signe) (on aurait pu faire autrement) :



FIGURE 1.24 – Dérivées secondes dans les sens des gradients, seuil : 0.02

On observe que c'est très bruité, sûrement à cause du léger bruit que possédait l'image original, accentué par les doubles dérivations. Appliquons un filtre gaussien en amont pour atténuer ce bruit :

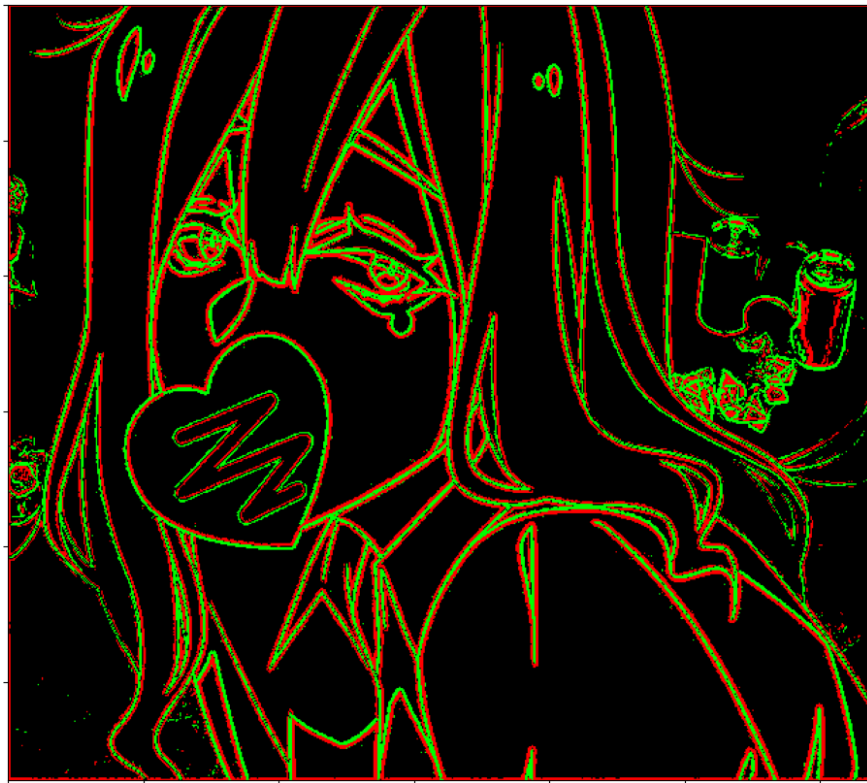


FIGURE 1.25 – Signes dérivées secondes dans les sens des gradients, en vert : positif, en rouge : négatif, seuil : 0.08. Prétraitement : filtre gaussien 3×3



FIGURE 1.26 – Dérivées secondes dans les sens des gradients, seuil : 0.08.
Prétraitement : filtre gaussien 3×3

On observe que cette méthode permet de détecter les contours autour des traits noirs présents sur l'image originale. Elle est donc plus précise que la première, en contrepartie d'être plus sensible au bruit.

1.6.2 Non annulation des déterminants des Hessiennes

On sait par le théorème de Schwarz que pour tout $(x, y) \in \mathbb{R}^2$, la hessienne de f en (x, y) , $\text{Hess}_f(x, y)$ est symétrique. Comme elle est en plus réelle, elle est diagonalisable. Et ses valeurs propres représentent les courbures maximales et minimales de f (à préciser)

On utilise un seuil aussi ici pour décider de dire si la Hessienne est nulle ou non. (on dit qu'on **binarise** l'image)

En vue de l'interprétation géométrique des valeurs propres des Hessiennes de f , on s'attend à ce que ce filtre détecte les contours de manière peu localisée et qu'il ne puisse pas détecter les contours droits horizontaux et droits verticaux.



FIGURE 1.27 – Déterminant de la Hessienne de f ,
seuil : 0.003

On observe que ça semble très instable, voyons ce que cela donne avec un seuil moins tolérant :



FIGURE 1.28 – Déterminant de la Hessienne de f , seuil : 0.001

Voyons ce que ça donne si on rajoute un peu de bruit :



FIGURE 1.29 – Déterminant de la Hessienne de f , seuil : 0.0035. Bruit gaussien, écart-type : 0.01

On peut pré-traiter l'image par un filtre passe-bas pour atténuer le bruit (même lorsqu'on n'en n'a pas ajouté) :



FIGURE 1.30 – Déterminant de la Hessienne de f , seuil : 0.0001. Prétraitement : filtre gaussien 3×3

On s'attendrait à que le filtre ne détecte que les contours courbés.
Il n'est pas très précis (contours épais).

1.6.3 Valeurs propres de la Hessienne

Les valeurs propres de la Hessienne donnent une indication sur la courbure de f autour de chaque point.

1.6.4 Ecart entre déterminant et trace de la Hessienne

plus il est grand plus on aurait de chance d'avoir une ligne?

1.6.5 Changement de signe du Laplacien

J'aime pas le laplacien

1.7 Ordre 3

Minimum plus bas ? mais sûrement trop sensible au bruit..

1.8 D'autres filtres

On notera abusivement que les filtres sont égaux à des fonctions (même si ça n'est qu'une version discrète). Dans le cas où le filtre a deux versions : horizontale et verticale, et que ces deux versions sont les mêmes à rotation de $\frac{\pi}{2}$ près, on omettra la version verticale.

Voici les images qui seront utilisées :

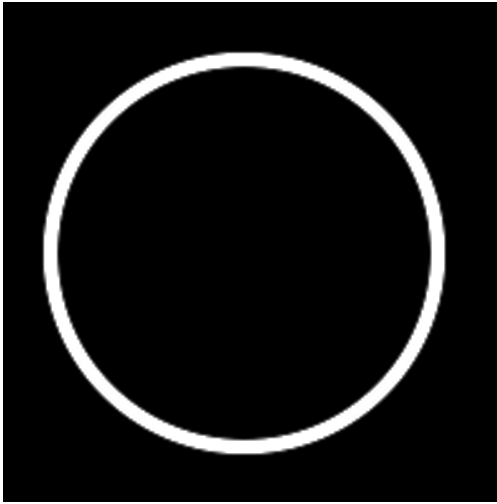


FIGURE 1.31 – Image originale

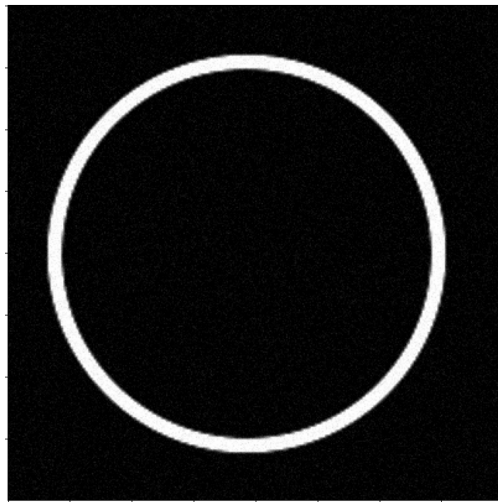


FIGURE 1.32 – Bruit gaussien, écart-type : 0.04



FIGURE 1.33 – Image originale

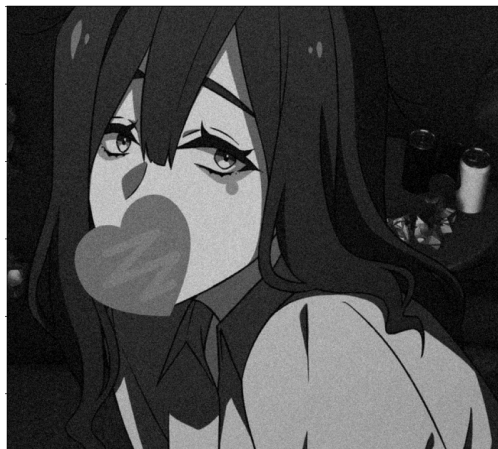


FIGURE 1.34 – Bruit gaussien, écart-type : 0.04



FIGURE 1.35 – Image originale



FIGURE 1.36 – Bruit gaussien, écart-type : 0.01

1.8.1 Filtres sans nom 1

Utile pour détecter des contours verticaux Deux versions :

$$\begin{pmatrix} 1 & 0 & 0 \\ 1 & \boxed{0} & -3 \\ 1 & 0 & 0 \end{pmatrix}$$

et

$$\begin{pmatrix} 1 & 0 & 0 \\ 1 & \boxed{-3} & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

Non séparable car de rang 2

Ces deux filtres sont très similaires et détectent les contours de la même manière, à l'exception que la deuxième version fournit des contours légèrement plus fins, ce qui est préféré.

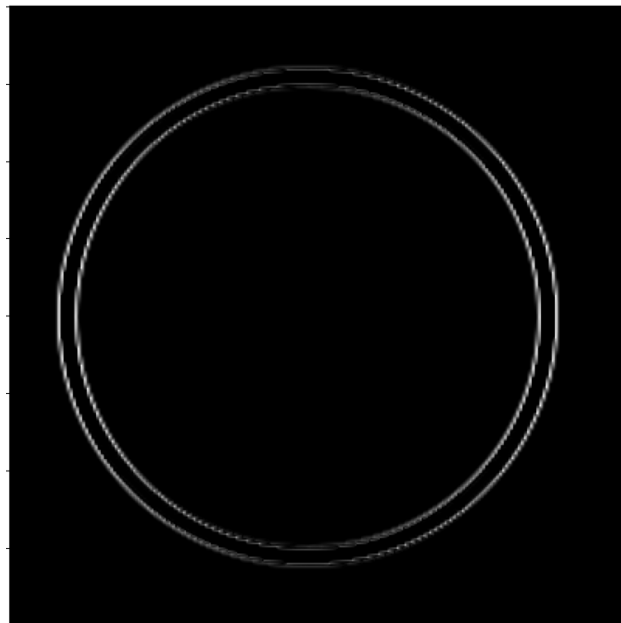


FIGURE 1.37 – Filtre 2, image normalisée

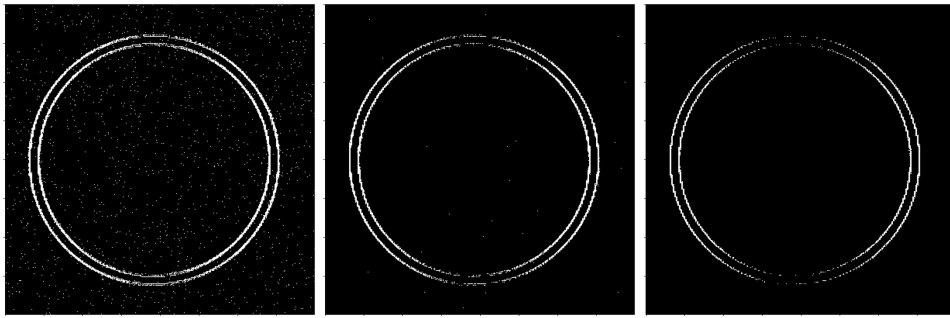


FIGURE 1.38 – Seuil de 0.25

FIGURE 1.39 – Seuil de 0.4

FIGURE 1.40 – Seuil de 0.55



FIGURE 1.41 – Filtre 2, image normalisée



FIGURE 1.42 – Seuil de 0.3

FIGURE 1.43 – Seuil de 0.4

FIGURE 1.44 – Seuil de 0.5



FIGURE 1.45 – Filtre 2, image multipliée par 2 pour plus de lisibilité



FIGURE 1.46 – Seuil de 0.7 FIGURE 1.47 – Seuil de 0.08 FIGURE 1.48 – Seuil de 0.12

1.8.2 Filtre de Sobel

C'est un filtre séparable, utile pour détecter les contours verticaux.

$$\begin{pmatrix} 1 & 0 & -1 \\ 2 & \boxed{0} & -2 \\ 1 & 0 & -1 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} * \begin{pmatrix} 1 & \boxed{0} & -1 \end{pmatrix}$$

On observe que $\begin{pmatrix} 1 & \boxed{0} & -1 \end{pmatrix} = 2 \frac{\partial f}{\partial x}$
et donc le filtre de Sobel est un flou suivi d'une dérivée première (DF) avec un facteur 2.

1.8.3 Filtre de Prewitt

Il est séparable, utile pour détecter les contours verticaux.

$$\begin{pmatrix} 1 & 0 & -1 \\ 1 & \boxed{0} & -1 \\ 1 & 0 & -1 \end{pmatrix} = \begin{pmatrix} 1 & \boxed{0} & -1 \end{pmatrix} * \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

gaussien
log
dog
Kayyali
Canny
Deriche

Differential
Robinson
Roberts cross
Maximally stable extremal regions

1.9 temp

1.10 Transformée de Radon

faire FT d'une Radon ?



FIGURE 1.49 – Image originale

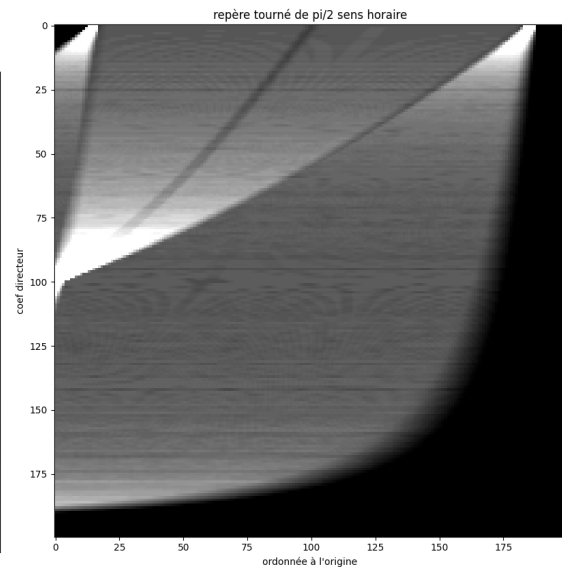


FIGURE 1.50 – Transformée de Radon de l'image (l'ordonnée des droites est une image par $\tan(\frac{\pi}{2} \cdot)$) à facteur près

1.11 Interpolation

1.12 Lignes de niveau et de courant

[geodiff analyse images et formes]

1.13 Transformée de Fourier

1.13.1 Continue 1D

On rappelle qu'une image analogique est une fonction $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ à support compact.

Définition 1.13.1 (FT continue, 1D). *Soit $f : \mathbb{R} \rightarrow \mathbb{R}$ à support compact. f est donc intégrable et on définit la transformée de Fourier (appelée FT) de f par :*

$$\mathcal{F}(f) : \begin{cases} \mathbb{R} \rightarrow \mathbb{R} \\ x \mapsto \int_{\mathbb{R}} f(t) e^{-i2\pi xt} dt \end{cases}$$

1.13.2 Discret 1D

Soit $T \in \mathbb{R}^+$ et $N \in \mathbb{N} \setminus \{0, 1\}$. On suppose :

$$\forall t \in \mathbb{R} \quad f(x + NT)$$

i.e. f est T -périodique.

On discrétise f via $(0, \frac{T}{N}, \frac{2T}{N}, \dots, \frac{(N-1)T}{N}) = (\frac{nT}{N})_{n \in \llbracket 0, N-1 \rrbracket}$.

On note pour $n \in \llbracket 0, N-1 \rrbracket$, $x_n := \frac{nT}{N}$

C'est-à-dire que l'on pose $\tilde{f} = f|_{\{x_0, \dots, x_{N-1}\}}$ la restriction de f à $(x_0, \dots, x_{N-1})$.

On cherche à exprimer $u := \begin{pmatrix} \tilde{f}(x_0) \\ \vdots \\ \tilde{f}(x_{N-1}) \end{pmatrix}$ dans la base $(v_j)_{j \in \llbracket 0, N-1 \rrbracket}$ définie comme suit :

$$\forall j \in \llbracket 0, N-1 \rrbracket \quad v_j := \begin{pmatrix} e^{ij \frac{2\pi \times 0}{N}} \\ e^{ij \frac{2\pi \times 1}{N}} \\ \vdots \\ e^{ij \frac{2\pi \times (N-1)}{N}} \end{pmatrix}$$

i.e. on cherche $\lambda_0, \dots, \lambda_{N-1} \in \mathbb{R}$ vérifiant :

$$u = \sum_{j=0}^{N-1} \lambda_j v_j$$

ce qui revient à écrire :

$$u = M \begin{pmatrix} \lambda_0 \\ \vdots \\ \lambda_{N-1} \end{pmatrix}$$

où $M := (v_0 \dots v_{N-1})$

Comme $(v_j)_j$ est une base, M est inversible et alors :

$$\begin{pmatrix} \lambda_0 \\ \vdots \\ \lambda_{N-1} \end{pmatrix} = M^{-1}u$$

On observe que les (v_j) sont orthogonaux 2 à 2 pour le produit hermitien canonique, et que pour $j \in \llbracket 0, N-1 \rrbracket$ on a :

$$\|v_j\|_2^2 = \sum_{k=0}^{N-1} \overline{[v_j]_k} [v_j]_k = \sqrt{N}$$

d'où :

$$M^{-1} = \frac{1}{\sqrt{N}} \overline{M}^\top$$

Et donc :

$$\begin{pmatrix} \lambda_0 \\ \vdots \\ \lambda_{N-1} \end{pmatrix} = \frac{1}{\sqrt{N}} \overline{M}^\top u$$

$$= \frac{1}{\sqrt{N}} \begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & e^{-i2\pi \frac{1 \times 1}{N}} & e^{-i2\pi \frac{1 \times 2}{N}} & \dots & e^{-i2\pi \frac{1 \times (N-1)}{N}} \\ \vdots & & & & \vdots \\ 1 & e^{-i2\pi \frac{(N-1) \times 1}{N}} & e^{-i2\pi \frac{(N-1) \times 2}{N}} & \dots & e^{-i2\pi \frac{(N-1) \times (N-1)}{N}} \end{pmatrix} u$$

Et on a obtenu une expression pour les λ_j . On déduit la définition suivante (on omet la normalisation) :

Définition 1.13.2 (FT discret, 1D). Soit $f : \mathbb{R} \rightarrow \mathbb{R}$ à support compact. On discrétise f via $(\frac{nT}{N})_{n \in \llbracket 0, N-1 \rrbracket}$. On définit la transformée de Fourier (appelée DFT) de f par :

$$\mathcal{DF}(\tilde{f}) : \begin{cases} \{x_0, \dots, x_{N-1}\} \rightarrow \mathbb{R} \\ x_k \mapsto \sum_{n=0}^{N-1} f(x_n) e^{-i2\pi \frac{nk}{N}} \end{cases}$$

1.13.3 Continue 2D

Définition 1.13.3 (FT continue, 2D). Soit f une image analogique. f est donc intégrable et on définit la transformée de Fourier de f par :

$$\mathcal{F}(f) : \begin{cases} \mathbb{R}^2 \rightarrow \mathbb{R} \\ x \mapsto \int_{\mathbb{R}^2} f(x) e^{-i\langle x, t \rangle} d\lambda_2(t) \end{cases}$$

où λ_2 est la mesure de Lebesgue sur $\mathbb{R}^2$

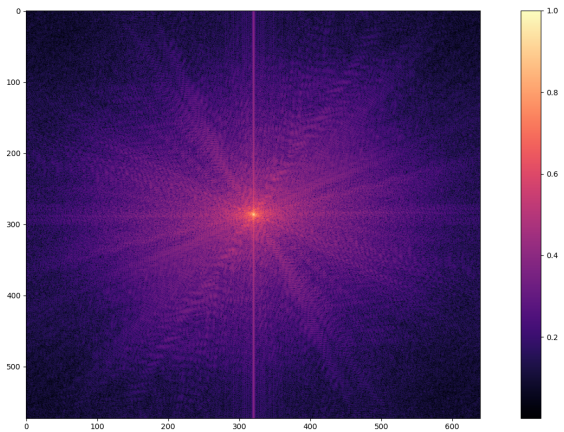
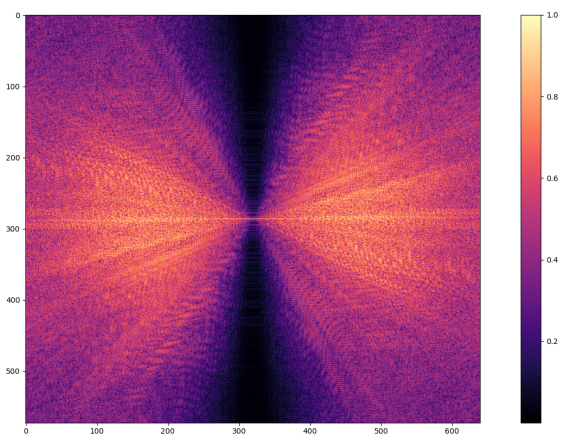
1.13.4 Discret 2D

Définition 1.13.4 (FT discret, 2D). Soient $M, N \in \mathbb{N}^*$. Soit $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ à support compact. On discrétise f via $(\frac{mT}{M}, \frac{nT}{N})_{n, m \in \llbracket 0, M-1 \rrbracket \times \llbracket 0, N-1 \rrbracket}$. On définit la transformée de Fourier (appelée DFT) de f par :

$$\mathcal{DF}(\tilde{f}) : \begin{cases} \llbracket 0, M-1 \rrbracket \times \llbracket 0, N-1 \rrbracket \rightarrow \mathbb{R} \\ (m_k, n_l) \mapsto \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m_k, n_l) e^{-i2\pi \frac{mk}{M}} e^{-i2\pi \frac{nl}{N}} \end{cases}$$



FIGURE 1.51 – Image originale

FIGURE 1.52 – Transformée de Fourier de l'image : Amplitude des coefficients (composées avec $\ln(1 + \cdot)$, puis normalisée par la norme sup de l'image)FIGURE 1.53 – Image filtré par DFD ($\frac{\partial f}{\partial x}$)FIGURE 1.54 – Transformée de Fourier de l'image : Amplitude des coefficients (composées avec $\ln(1 + \cdot)$)

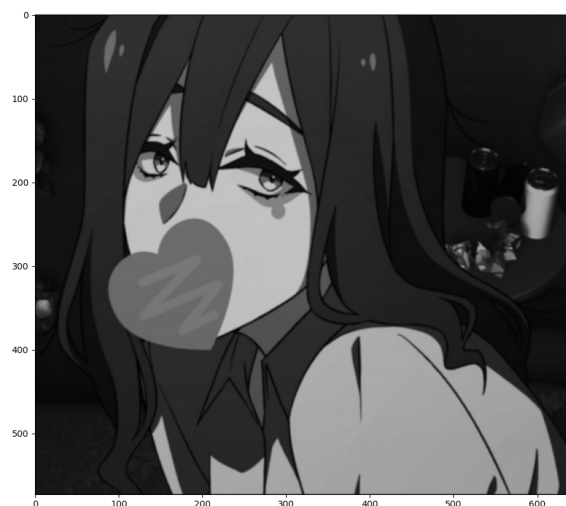


FIGURE 1.55 – Image filtré par filtre gaussien 3×3

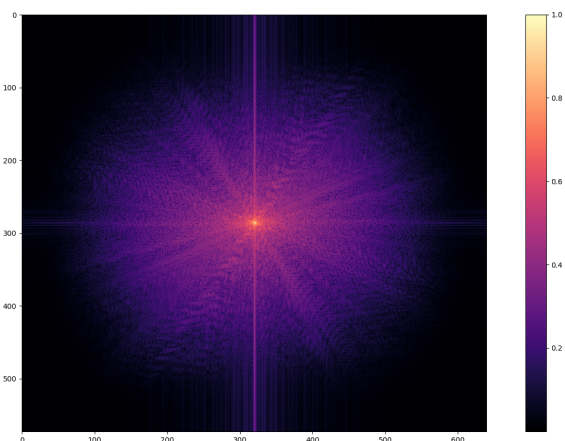


FIGURE 1.56 – Transformée de Fourier de l'image : Amplitude des coefficients (composées avec $\ln(1 + \cdot)$)

1.14 Retour sur la convolution

transformée de fourier : point de vue fréquentiel

on peut voir un filtre comme une application linéaire (car convolution linéaire)

Définition 1.14.1 (Correspondance fréquentielle de la convolution). *Soit u une image et h un filtre. Alors*

$$\mathcal{F}(u * h) = \mathcal{F}(u) \times \mathcal{F}(h)$$

1.15 Convolution non linéaire

Filtre bilatéral

filtre toboggan

filtre médian (trop fort pour denoise)

1.16 Transformée d'ondelettes

On part du principe que plus les fréquences sont basses, plus elles sont là tout le temps.

"Analyse multirésolutionnelle"

1.16.1 Transformée d'ondelettes continue

Définition 1.16.1. Soit $\psi \in L^2(\mathbb{R})$. On pose pour $(m, n) \in \mathbb{Z}^2$:

$$\psi_{m,n} : t \mapsto \frac{1}{\sqrt{2^m}} \psi \left(\frac{t - 2^m}{2^m} \right)$$

Définition 1.16.2. Soit $\psi \in L^2(\mathbb{R})$. On dit que ψ est une ondelette si $(\psi_{m,n})_{(m,n) \in \mathbb{Z}^2}$ est une base Hilbertienne de $L^2(\mathbb{R})$.

Dans ce cas on appelle ψ une **ondelette mère**.

Définition 1.16.3 (Transformée d'ondelettes). Soit ψ une ondelette. On définit la transformée d'ondelettes de f par ψ par :

$$\mathcal{W}_\psi(f)(a, b) = \langle f, \psi_{a,b} \rangle$$

où $\langle \cdot, \cdot \rangle$ désigne le produit scalaire canonique de $L^2(\mathbb{R})$

Exemples d'ondelettes (mères) :

Ondelette de Haar :

$$\mathbb{1}_{[-1,0]} - \mathbb{1}_{[0,1]}$$

Ondelette Sombrero

Ondelette de Daubechies

Ondelette de Coiflet

1.16.2 Transformée d'ondelettes discrètes

A 1 dimension

Définition 1.16.4. Soit $\psi \in L^2(\mathbb{R})$. On dit que ψ est une ondelette si $(\psi_{m,n})_{(m,n) \in \mathbb{Z}^2}$ est une base Hilbertienne de $L^2(\mathbb{R})$.

Dans ce cas on appelle ψ une **ondelette mère**.

Définition 1.16.5. Soit $n \in \mathbb{N}^*$. Soit $(t_1, \dots, t_n) \in \mathbb{R}^n$. Soit $f \in L^2(\mathbb{R})$. On appelle **transformée en ondelette discrète** de f l'application :

$$D_f : \begin{cases} \mathbb{R} \times]0, +\infty[\rightarrow \mathbb{R} \\ (a, b) \mapsto \frac{1}{\sqrt{b}} \sum_{k=1}^n f(t_k) \psi \left(\frac{t_k - a}{b} \right) \end{cases}$$

où $a = k2^{-j}$ et $b = 2^{-j}$

1.17 A s'intéresser

Ridge detection

Hough transform

Hessian affine region detector

SIFT

reconnaissance d'instances d'objets

plot les valeurs propres de la hessienne sur une ligne (en considérant l'image quand même, pour avoir deux valeurs propres)

[1]

Bibliographie

- [1] Fanny Delebecque. Projet : “traitement d’images, détection de contours”. Université de Toulouse III, FSI – Département de Mathématiques Licence 2, module “Calcul scientifique”, 2019–2020, <https://perso.math.univ-toulouse.fr/fdelebec/files/2019/02/image.pdf>.